



第2章 软件生命周期

主要内容

1. 项目构想和立项
2. 可行性分析
3. 项目开发计划
4. 需求分析
5. 软件设计
6. 软件实现
7. 软件测试
8. 软件维护
9. 小结



1. 项目构想和立项

对软件的工作范围进行分析，寻找实现软件的可行解决方案，预测软件或应用的发展方向，为后期的软件开发做准备。



1. 项目构想和立项

项目构想阶段的最终成果是问题定义报告，其主要内容包括：

- [1] 待开发工程项目的名称
- [2] 软件项目的使用单位和部门
- [3] 软件项目开发单位
- [4] 问题的概括定义
- [5] 软件项目的用途和目标
- [6] 软件项目的类型和规模
- [7] 软件按项目开发的开始时间以及预计交付使用时间
- [8] 软件项目开发可能投入的经费
- [9] 软件项目使用单位和开发单位双方确认信息等

1. 项目构想和立项

如何建设空中楼阁?



2.可行性分析

针对项目构想的内容进行详细分析，主要包括：

- [1] 技术可行性
- [2] 经济可行性
- [3] 操作可行性
- [4] 社会可行性

同时，可行性分析也必须分析项目是否值得完成。

2.可行性分析

[1] 技术可行性

从技术的角度出发，分析实现、完成软件的可行性。



2.可行性分析

[1] 技术可行性

资源分析

评估现有软硬件资源是否能够满足待实现目标系统所需的条件，是否满足待开发系统所需要的要求。

技术分析

对当前已有的技术，或者对开发单位拥有的技术进行评估，分析所选技术是否先进、合理，以及技术人员的能力水平和工作基础是否能达到需要的水平。

效率分析

评估在给定的人员、资金、设备、时间情况下，评估以现有的技术和开发方法是否可以设计出目标系统，并实现所必需的功能和性能。

2.可行性分析

[2] 经济可行性

从经济角度出发，确定软件项目是否值得开发，帮助用户做出是否投资于此工程开发的决定。

经济可行性分析主要包括：

- ✓ 成本估计
- ✓ 效益估计

2.可行性分析

[2] 经济可行性—成本估计

一般而言，软件的成本主要包括硬件成本、开发费用、施工费用、系统运行费用、维护费用和人员培训费用等。

运营成本则是指软件项目的运行、管理和维护所花费的费用。

经典的软件开发的工作量估算方法：

- ✓ 代码行估计
- ✓ 功能点技术度量

2.可行性分析

[2] 经济可行性—效益估计

软件的效益分为直接效益和间接效益两个方面。

直接效益

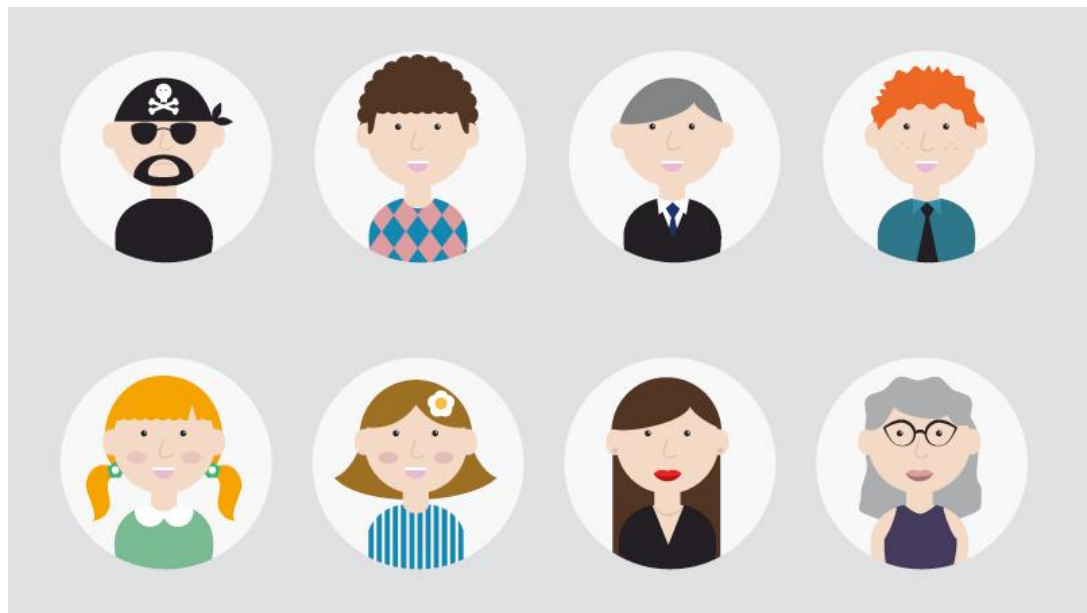
间接效益

- ✓ 直接效益是软件项目直接获取的效益或者节省的运行费用
- ✓ 间接效益是指能够提高企业信誉和形象，但不能用资金计算的效益

2.可行性分析

[3] 操作可行性

主要考虑目标软件的使用和操作方式是否符合用户的技术水平和使用习惯。



目标用户



软件的运行和使用方式

2.可行性分析

[4] 社会可行性

分析和论证目标软件系统在当前的法律和道德情况下，其运行方式、操作规程在用户组织或者环境内是否合理、合法，以及是否存在违规、侵权等一系列问题。

社会可行性分析至少需要包括：

- ✓ 市场政策
- ✓ 法律问题

3.软件开发计划

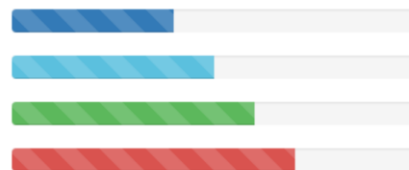
对参与项目的人员、组织、进度、资金、设备、时间、资源等因素进行合理的规划，统筹管理整个项目的实施过程。



负责人员



成本



进度



需要的软硬件

3.软件开发计划

阶段划分



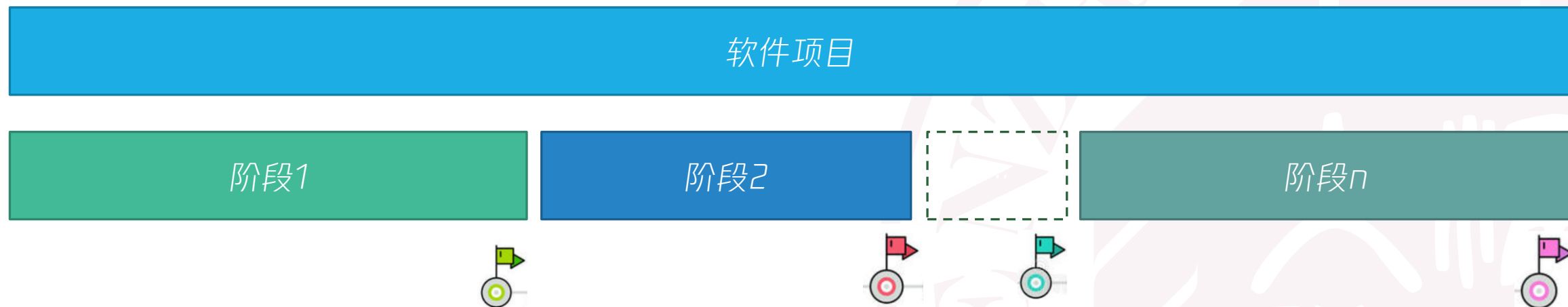
3.软件开发计划

内容划分



3.软件开发计划

里程碑划分



3.软件开发计划

资源划分—时间、人员、设备、资金等



3.软件开发计划

当软件开发计划制定完成后，开发组织还需要对完成的项目开发计划进行评审，对不合理的计划内容进行修改或重新制定。

软件开发计划细化，如测试计划、分包商计划、风险管理计划、评审计划和验收计划等。

3.软件开发计划

[1] 制定软件开发计划

在可行性分析报告的基础上，由软件开发人员和用户对项目在不同开发阶段的工作量和所需时间进行估算，通过对各种资源进行规划，争取在交付日期前完成软件项目的开发。

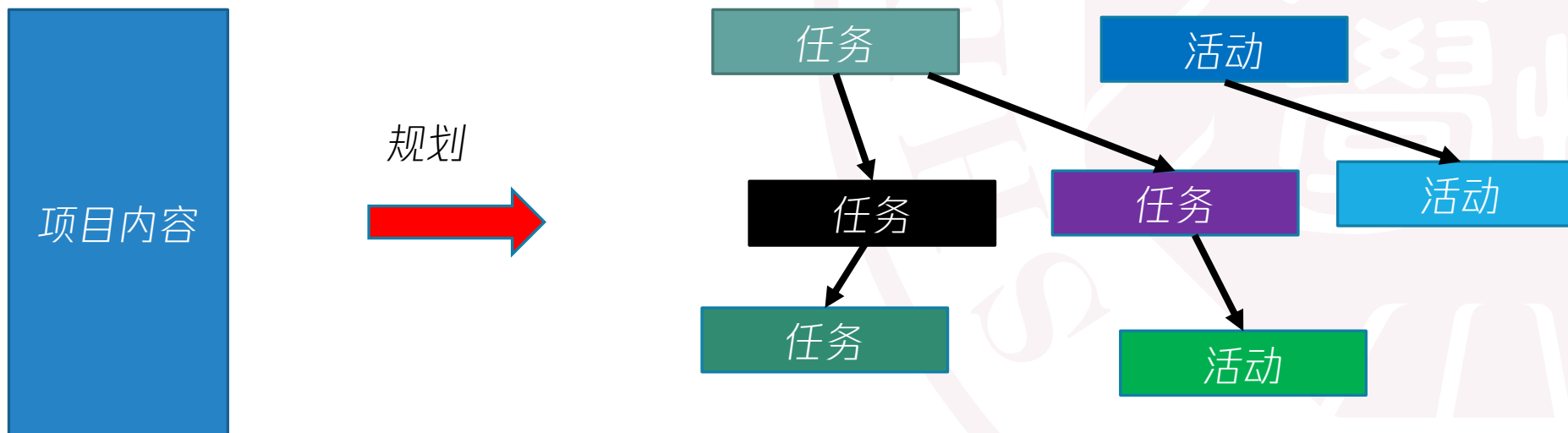
同时，考虑目标系统软硬件资源的购买、设置及分配等。

3.软件开发计划

[1] 制定软件开发计划

① 项目的内容范围

确定项目所涉及的工作内容和范围，对软件的功能、性能、可靠性等形成一个总的说明，作为整个项目开发计划的设定基础。



3.软件开发计划

[1] 制定软件开发计划

② 资源计划

a.人力资源:

结合项目需要对可获得的人员资源进行规划，不同阶段安排不同的人员，以及协调不同人员之间的组织关系。

b.软、硬件资源

对开发项目的不同任务或者活动对软硬件的需求关系进行分析，在规划项目任务和活动时加入软硬件资源的可用性分析。

c. 时间资源

结合各项任务或者活动的工作量，以及安排的各项资源，为各个任务或者活动规划项目开发时间。

3.软件开发计划

[1] 制定软件开发计划

③ 项目开发进度安排

项目的开发进度安排需要综合考虑各种情况,从各种角度将各种资源规划到最佳。

在安排软件开发进度时, 需要重点考虑:

- ✓ 开发进度与开发人员的数量关系
- ✓ 开发进度与开发人员的合理分配
- ✓ 工作量和任务之间的衔接关系和要求

每项任务完成后, 应该以交付的文档和复审通过为完成标准, 确定好各个阶段的标志性里程碑。

3.软件开发计划

[1] 制定软件开发计划

项目开发计划一般包括项目概述、项目实施计划、项目人员配置和项目交付日期等内容。

- ✓项目概述：说明软件的功能、类型、性能、项目程序所使用的语言、存储形式。
- ✓项目实施计划：说明各阶段的任务、进度，各项任务的负责人，项目各阶段的预算。
- ✓项目人员配置：说明各阶段项目所需人员的数量和类型。
- ✓项目交付日期：说明项目的交付日期。

3.软件开发计划

[2] 项目开发计划评审

在项目开发计划完成后，需要对完成的计划内容进行多次评审，对项目开发计划中的进度安排，资源分配等进行评判。通过多次评审和修订后，形成最终的项目开发计划文档。



4.需求分析

[1] 需求的定义与和组成

需求分析是发现、求精、建模、规格说明和复审的过程。

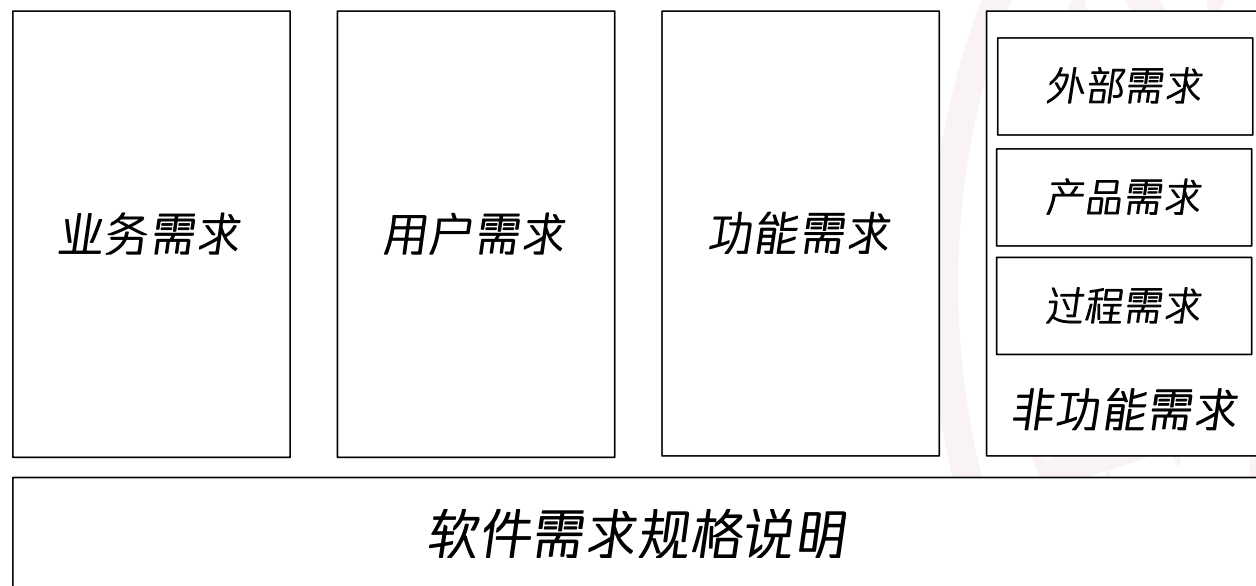
需求分析也是从宏观角度调查、分析用户面临的问题，然后从微观角度分析并描述用户实际需求的过程。

需求分析的定义 [IEEE 1997年发布的《软件工程结构标准词汇表》]：

- 需求分析可以理解为用户解决问题或者达到目标所需的条件或能力；
- 需求分析可以是系统或者系统部件为满足合同、标准、规范或其他正式文档所需具有的条件或能力；
- 需求分析是一种反映上述1和2条件或者能力的文档描述。

4.需求分析

一般而言，软件需求包括业务需求、用户需求、功能需求和非功能需求四个层次。



4.需求分析

① 业务需求

立项项目的主要研发目的，是开发软件系统的主要目标。

② 用户需求

指除业务需求以外，用户希望立项项目中提供的其他需求。

③ 功能需求

指在完成项目业务需求和用户功能需求时，软件系统需要提供的其他系统功能。

4.需求分析

④ 非功能性需求

从各个角度对系统的约束和限制，反映了用户对软件系统质量和特性的额外要求。

✓ 过程需求

- 对系统交付的方式、实现技术细节和实现标准进行要求，如操作系统类型、程序设计语言选择、数据库选择等；

✓ 产品需求

- 对目标软件系统的性能、可用性、实用性、可靠性、可移植性、安全保密性、容错性等方面进行约束；

✓ 外部法规需求

- 对软件产品需要遵循的法规、成本、操作性、开发时间等进行了约束。

4.需求分析

[2] 需求管理

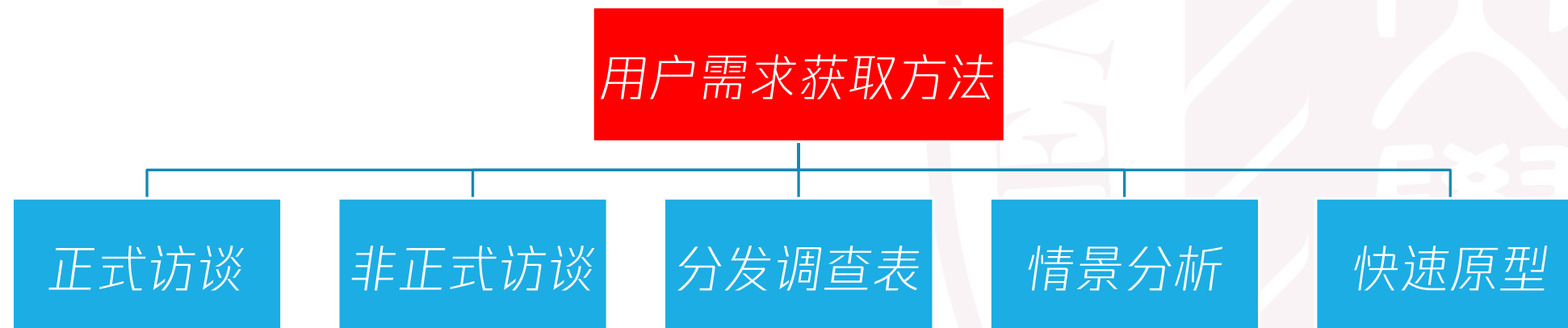
从项目管理角度而言，需求的开发和管理过程是用来获取、导出、确认和维护需求的一系列行为的组合。



4.需求分析

① 需求获取

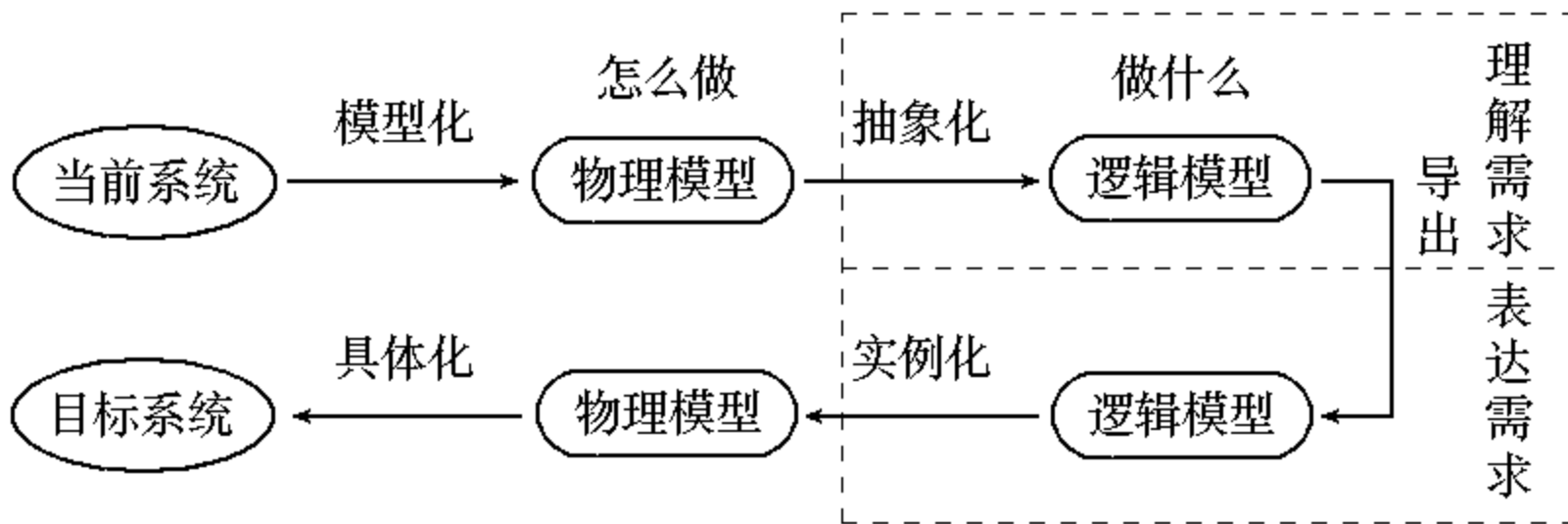
通过需求调研，获得清晰、准确的用户需求的过程。通过对即将实验的项目需求进行了解，准确表达用户对项目的需求内容。



4.需求分析

② 需求建模

对用户的需求内容进行建模, 保证需求在描述和理解过程中不会失真。



4.需求分析

③ 需求规格说明

描述软件系统必须实现的软件功能和性能，对软件必须实现的功能和性能，以及对系统开发约束进行了描述。

- ✓ 作为用户和开发方之间的约束或者合同内容;
- ✓ 反映问题的结构，作为系统设计和编码的依据;
- ✓ 作为测试和验收目标系统的依据。

4.需求分析

④ 需求评审

需求评审是对完成需求质量的确认过程。

可以发现需求规格说明书中的错误或者遗漏，并进行修正。

确保需求规格说明书中描述的是用户真实的需求，避免因需求不准确带来的需求变更，降低项目开发风险。



软件需求评审指标点

4.需求分析

⑤ 需求变更

需求管理也是对需求的内容进行控制的过程。

由于软件需求是项目开发、测试的基础，软件需求的变更将对项目带来较大的影响。因此，必须有效地管理需求的变更，降低需求变更对项目带来的风险。

4.需求分析

一般而言，需求变更需要经过评审、裁定和实施三个步骤。



4.需求分析

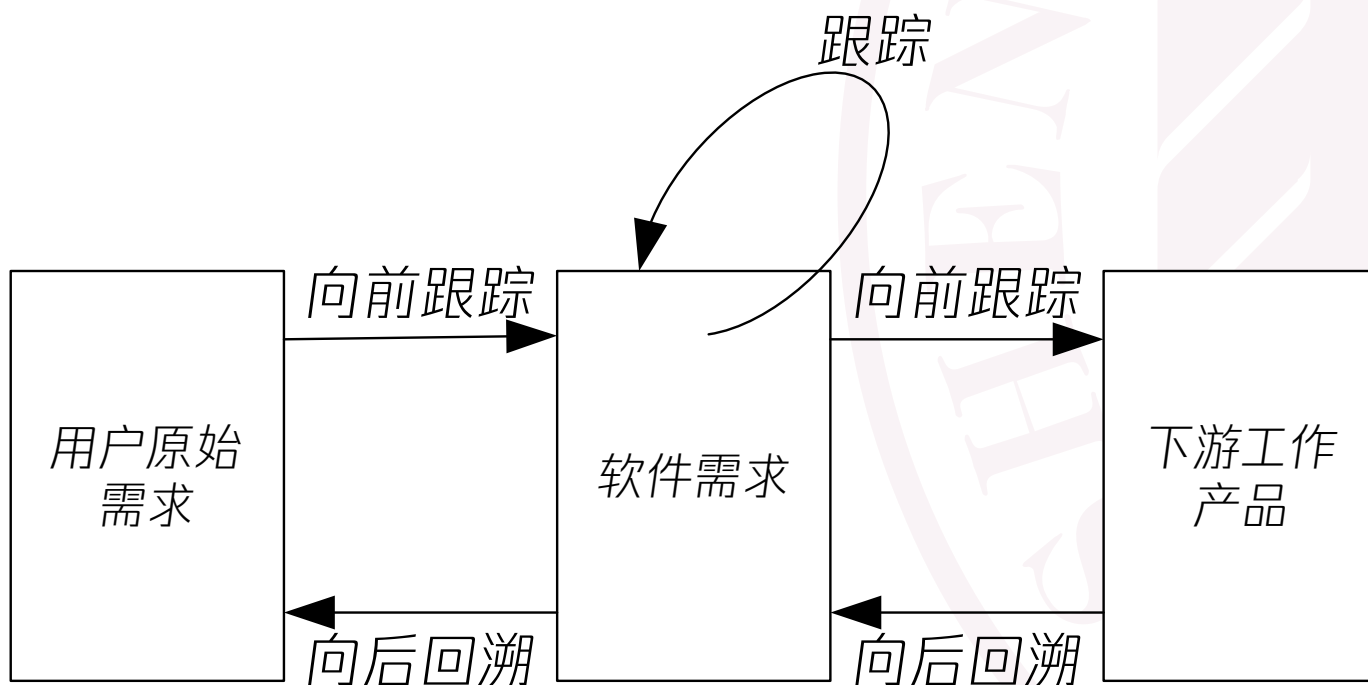
需求管理需要注意的内容:

- ✓ 为了保证需求版本管理的有效性, 必须为需求建立严格的版本管理制度;
- ✓ 建立明确的需求变更控制流程;
- ✓ 需求变更需要通知所有涉及的相关人员;
- ✓ 及时更新受影响的软件开发计划、相关活动等。

4.需求分析

⑥ 需求跟踪

将软件开发中的各个需求与其他系统元素之间的依赖关系和逻辑联系建立跟踪，有助于开发者全面分析变更带来的影响，并作出正确的变更决策。



4.需求分析

通过需求矩阵将各种类型的需求，业务规则，系统架构，设计组件，源码，测试用例，以及帮助文档等进行挂钩。

需求	其他需求	业务规则	系统架构	设计组件	源码	测试用例	帮助文档
<i>R1</i>	<i>R5,R6</i>	<i>B1,B3</i>	<i>T1,T3</i>	<i>D1,D3,D3.1</i>	<i>S1,S3,S31</i>	<i>U1,U3</i>	<i>H1,H3</i>
<i>R2</i>	<i>R3,R6</i>	<i>B5</i>	<i>T3</i>	<i>D3,D6</i>	<i>S3,S6.1</i>	<i>U3,U6,U7</i>	<i>H1,H6</i>
<i>R3</i>	<i>R7,R5</i>	<i>B1,B6</i>	<i>T5,T4</i>	<i>D4,D5</i>	<i>S4,S5</i>	<i>U2,U5,U6</i>	<i>H2,H6</i>
...	...	...	...	...	...	...	...

4.需求分析

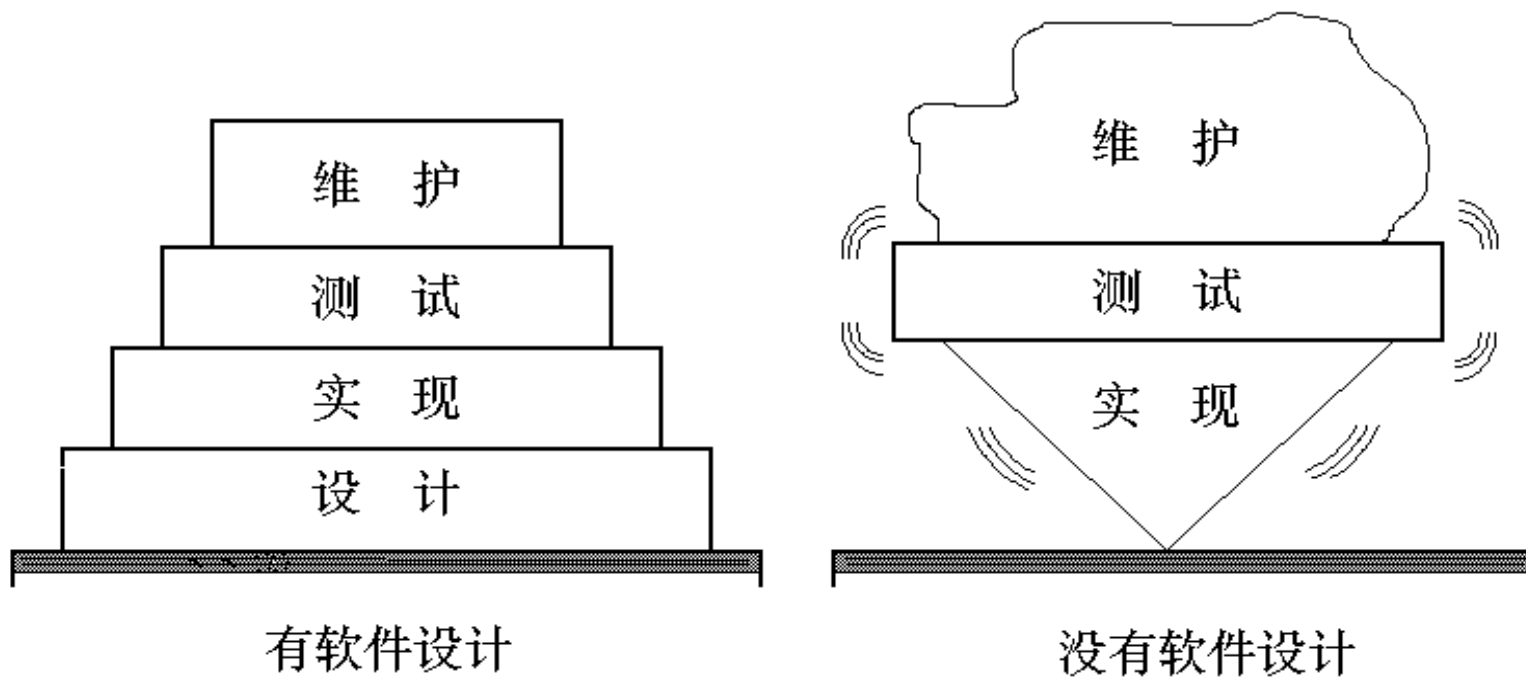
⑦ 需求状态跟踪

除了对需求进行跟踪以外，还需要对需求的状态进行跟踪。

通常而言，需求具有多种属性，如状态、优先级、稳定性、创建时间、创建人等。

结合不同的工作情况，需求的状态有已建议、已批准、设计中、实现中、已实现、测试中、已验证等多种形式。

5. 软件设计



软件设计的重要性

5. 软件设计

[1] 软件设计的目标和任务

依据需求分析内容，完成适合开发目标软件系统的体系结构、数据描述、软件构件及接口，以及软件系统组成等相关内容的设计。

5. 软件设计

[2] 软件设计阶段

从工程管理的角度而言，软件设计可以分为概要设计和详细设计两个阶段。

① 概要设计 [也称为总体设计]

从整体上对目标软件系统的结构或者架构进行设计说明。

② 详细设计

以概要设计为基础，对概要设计中规划的各项内容的进一步细化。

5. 软件设计

结构化方法概要设计

划分功能模块

确定模块之间的层次结构

确定模块之间的调用关系

确定模块之间的接口参数传递信

5. 软件设计

面向对象概要设计

确定目标系统的架构设计

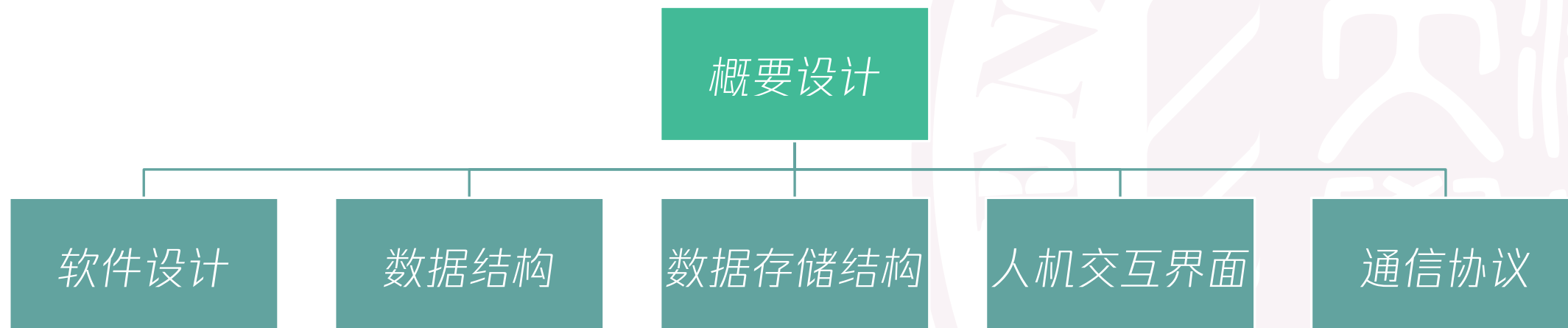
确定每个类的属性和方法

确定构件提供的接口等

确定各个类、构件之间的调用关系

5. 软件设计

概要设计阶段还需要确定软件系统中使用的数据结构、数据存储结构等。同时，软件应用系统与操作用户、通信程序之间的接口交互方式也应当在本阶段确定下来。



5. 软件设计

结构化方法详细设计

系统部署

局部数据结构

代码文件组织

模块使用的算法

5. 软件设计

面向对象详细设计

进一步精化软件架构

给出类所包含的属性和方法实现细节

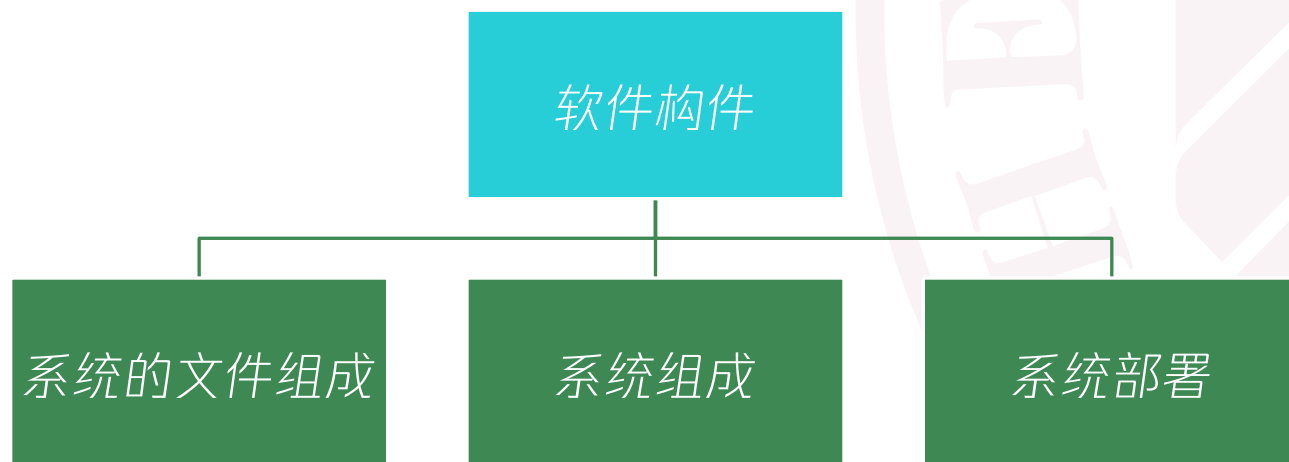
提供多个类协同完成用例的方法

确定不同对象、构件之间的消息互动等。

5. 软件设计

当系统涉及到构件模块时，系统的文件组成、系统组成和部署等内容也需要同步完成。

如果目标系统中涉及到数据系统〔数据库或大数据〕，详细设计阶段还需要完成与数据相关的各项设计工作。



5. 软件设计

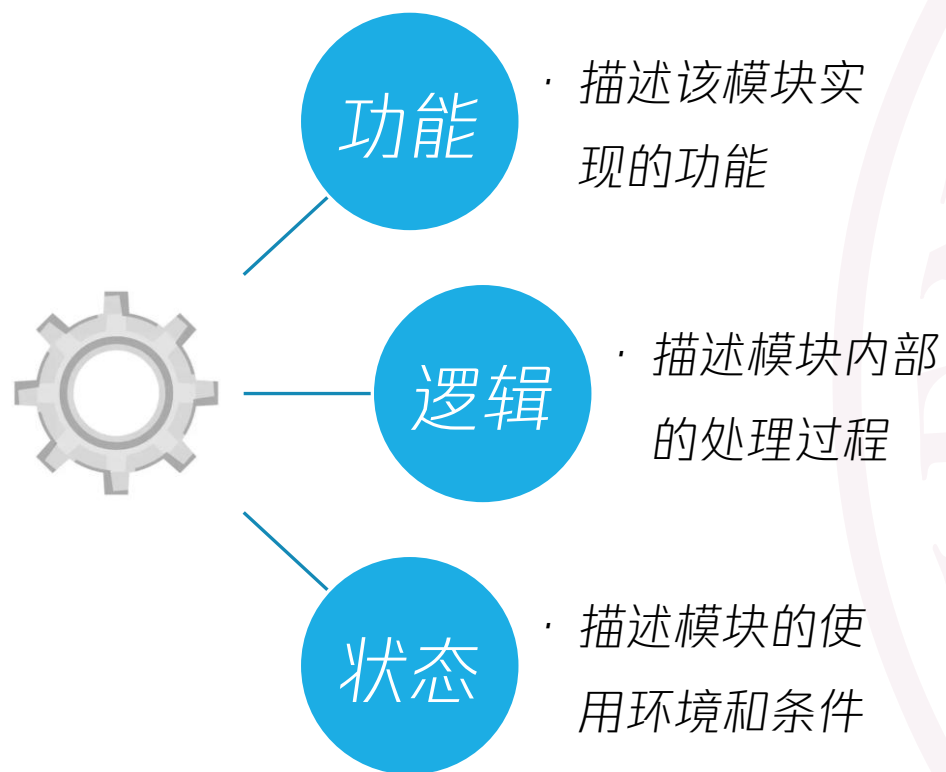
[3] 模块化与模块独立

模块化是指软件设计人员通过“分而治之”方法将软件内容划分为若干个独立的模块，通过将多个完成特定功能的模块内容进行集成，实现软件需求的过程。



5. 软件设计

模块具有功能、逻辑和状态三个基本属性。



5. 软件设计

软件设计从“耦合”和“内聚”两个角度来度量模块的独立性。

耦合： 对软件结构内不同模块之间互连程度的度量；

内聚： 对模块内各个元素彼此结合的紧密程度〔模块功能强度〕的度量。

5. 软件设计

结构化设计的模块独立性

结构化模块： 实现特定功能的过程函数、子程序或者宏定义；

耦合： 函数与函数之间的连接程度

内聚： 函数内部所涉及业务逻辑的紧密程度

5. 软件设计

面向对象设计的模块独立性

面向对象模块：组成目标系统的类或者构件

耦合：类与类之间，或者对象与对象之间为完成业务功能的消息交互密切程度；

内聚：类或者构件中封装的属性和方法的紧密程度。

5. 软件设计

[4] 设计复审

通过软件设计复审，软件开发团队可以尽早发现并排除软件设计中存在的缺陷，减少软件设计缺陷带来的影响。

软件设计复审可以分为对概要设计的复审和对详细设计的复审。

5. 软件设计

结构化设计复审

概要设计复审主要关注

- ✓ 系统的体系结构
- ✓ 模块划分
- ✓ 模块接口
- ✓ 内外接口等

详细设计复审主要关注

- ✓ 各个软件模块的实现细节
- ✓ 模块采用的算法
- ✓ 模块采用的数据结构
- ✓ 模块的性能等。

5. 软件设计

面向对象设计复审

概要设计复审主要关注

- ✓ 软件系统的架构设计
- ✓ 类设计是否合理;
- ✓ 类属性、状态是否合理等

详细设计复审主要关注

- ✓ 实现软件业务的对象交互、软件构件、软件部署模式等内容是否合理

6. 软件实现

软件实现是指软件开发团队利用程序设计语言，将详细设计的成果转换为软件系统，并对完成的软件系统进行质量保障的过程。

6. 软件实现

[1] 程序设计语言的选择

通常情况下，程序设计语言的选择对软件实现没有太大影响。但是，在某些特殊的情况下，软件开发团队必须结合项目需求，以及解决问题的实际情况为目标项目选择合适的程序设计语言。

优秀的程序设计语言应当具备的特点：

- ✓ 具有理想的模块化机制
- ✓ 拥有可读性好的控制结构和数据结构
- ✓ 编译程序能尽可能多地发现程序中的错误
- ✓ 具有良好的独立编译机制

6. 软件实现

在实际选择程序设计语言时，还必须考虑实用方面的各种限制。

- ✓ 系统用户的要求
- ✓ 可以使用的编译程序
- ✓ 可以得到的软件工具
- ✓ 工程规模和复杂程度
- ✓ 软件的应用领域
- ✓ 软件的可移植性要求
- ✓ 程序员的知识

6. 软件实现

[2] 软件编码规范

软件编码规范是指软件开发团队或者开发组织为了规范软件开发人员的编码风格和编程习惯所定义的软件代码编写规则。

空行

空格

排版规范

代码注释

代码行

合理使用括号

标识符命名与
定义

目录结构规范

文件命名规范

6. 软件实现

[3] 程序效率

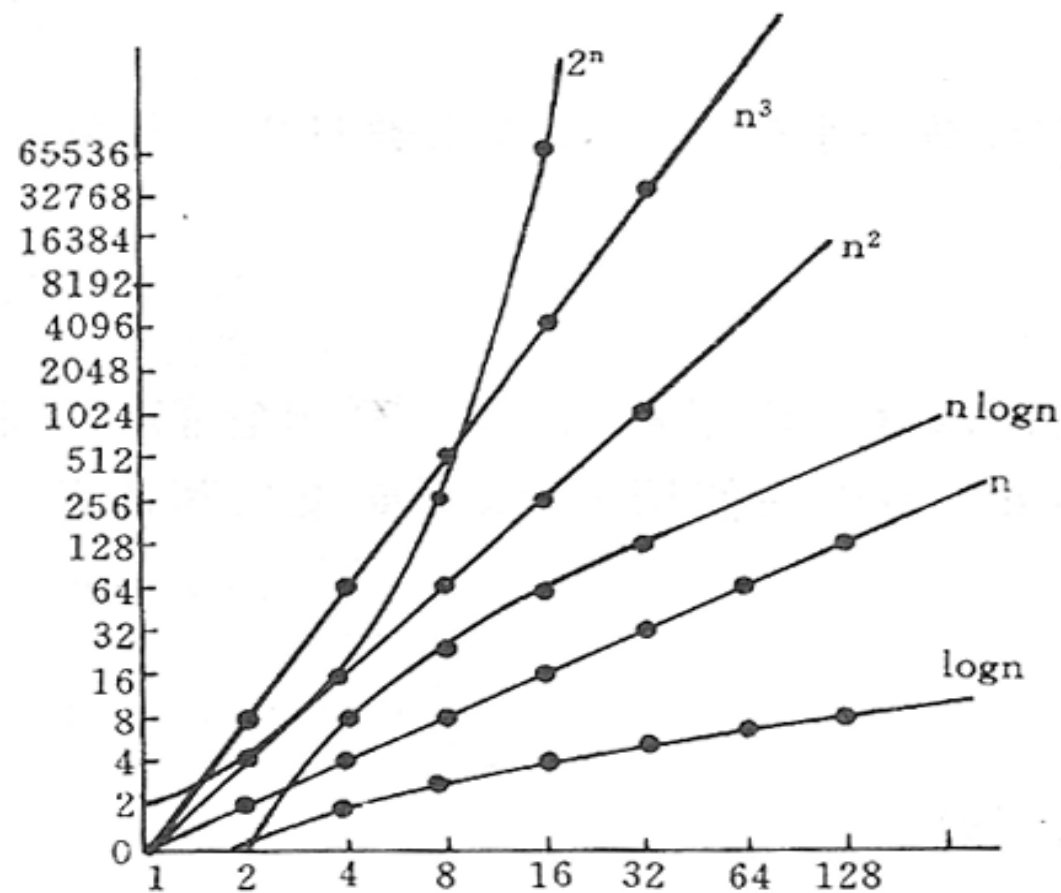
软件的效率可以分为全局效率、局部效率、时间效率和空间效率四个方面。

- 全局效率：从整个系统的角度来评估软件的效率；
- 局部效率：从软件模块或函数角度上来评价软件的效率；
- 时间效率：衡量软件处理任务所需的时间长短；
- 空间效率：对软件运行时所需内存空间的衡量等。

6. 软件实现

软件开发团队可以参考以下原则来提高目标软件的效率：

- ✓ 选择合适的数据结构
- ✓ 选择优化的算法
- ✓ 提高存储效率
- ✓ 优化输入/输出效率
- ✓ 提高程序的运行效率



7. 软件测试

软件测试是一种用来促进鉴定软件的正确性、完整性、安全性和质量的过程。

使用人工或自动的手段来运行或测定某个软件系统的过程，其目的在于检验它是否满足规定的需求或弄清预期结果与实际结果之间的差别。



输入数据



待测软件

输出数据



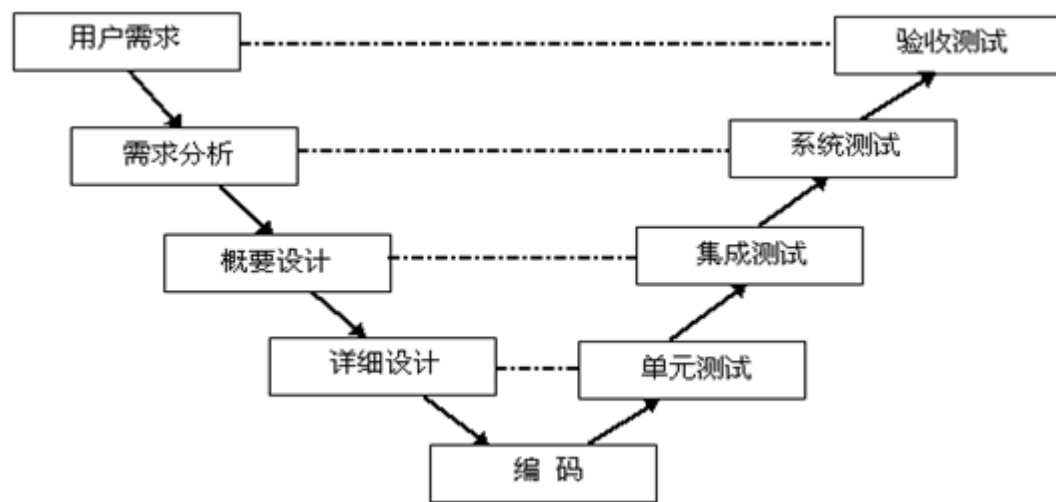
7. 软件测试

从不同的立场出发，软件测试的目标是不同的。

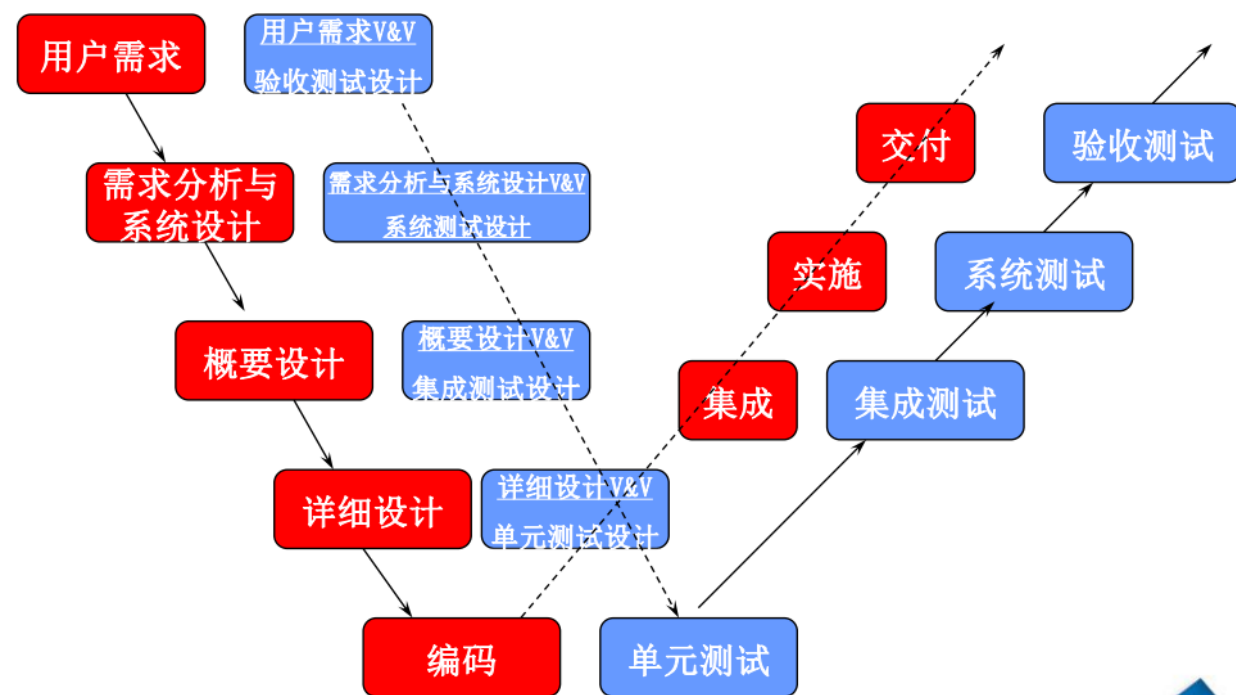
- ✓ 用户希望通过软件测试暴露软件中隐藏的错误和缺陷，以考虑是否可接受该产品；
- ✓ 软件开发者希望测试成为表明软件产品中不存在错误的过程，验证该软件已正确地实现了用户的要求，确立人们对软件质量的信心。

7. 软件测试

随着计算机技术和软件技术的飞速发展，软件测试技术研究也取得了较大的突破。通过总结多年的软件测试经验，测试专家总结出了大量优秀的测试模型，例如著名的V模型、W模型等。



V模型



W模型

7. 软件测试

软件测试包括制定软件测试计划、设计测试用例、测试准备、执行测试、测试结果评估和缺陷跟踪等阶段，并且每个阶段均有相应的完成标准。



7. 软件测试

[1] 软件测试技术

如何为被测软件找到合适的测试用例是软件测试的关键。



7. 软件测试

根据软件测试用例的设计依据不同，软件测试用例的设计方法可以分为**白盒测试**、**黑盒测试**和**性能测试**三种类型。

① 白盒测试

白盒测试，又称结构测试，也称为基于代码的测试或者逻辑驱动测试。

白盒测试是在软件结构或者程序代码已知的情况下，软件测试人员利用程序内部的逻辑结构及相关信息来设计或选择测试用例，检验程序是否按照要求正常运行的测试过程。

7. 软件测试

根据软件测试人员是否在计算机上执行被测试程序，白盒测试可以进一步分为**静态白盒测试**和**动态白盒测试**两种。

➤ 静态白盒测试

主要依靠人工审查方式对完成的软件设计、软件体系结构和代码进行检验，力争发现软件中可能存在的错误。

- 正式的代码审查
- 结对编程
- 轻量型的非正式代码审查

7. 软件测试

➤ 动态白盒测试

根据程序的内部结构、处理逻辑等因素来设计软件测试用例，并利用构造的测试用例来运行被测软件，按照一定的策略发现软件代码中存在的错误和缺陷的过程。



7. 软件测试

② 黑盒测试

又称功能测试，也称基于规格说明的测试或数据驱动测试。

根据软件的需求规格说明和软件使用说明来设计或选择测试用例，测试软件实现的功能是否与需求规格说明书相符。

由于程序的内部结构不透明，软件测试人员只能通过软件的外部操作和数据输入来检测软件内部的功能和业务是否符合需求。

7. 软件测试

常见的黑盒测试用例设计方法

等价类划分法

边界值分析法

因果图法

判定表驱动法

错误推测法

7. 软件测试

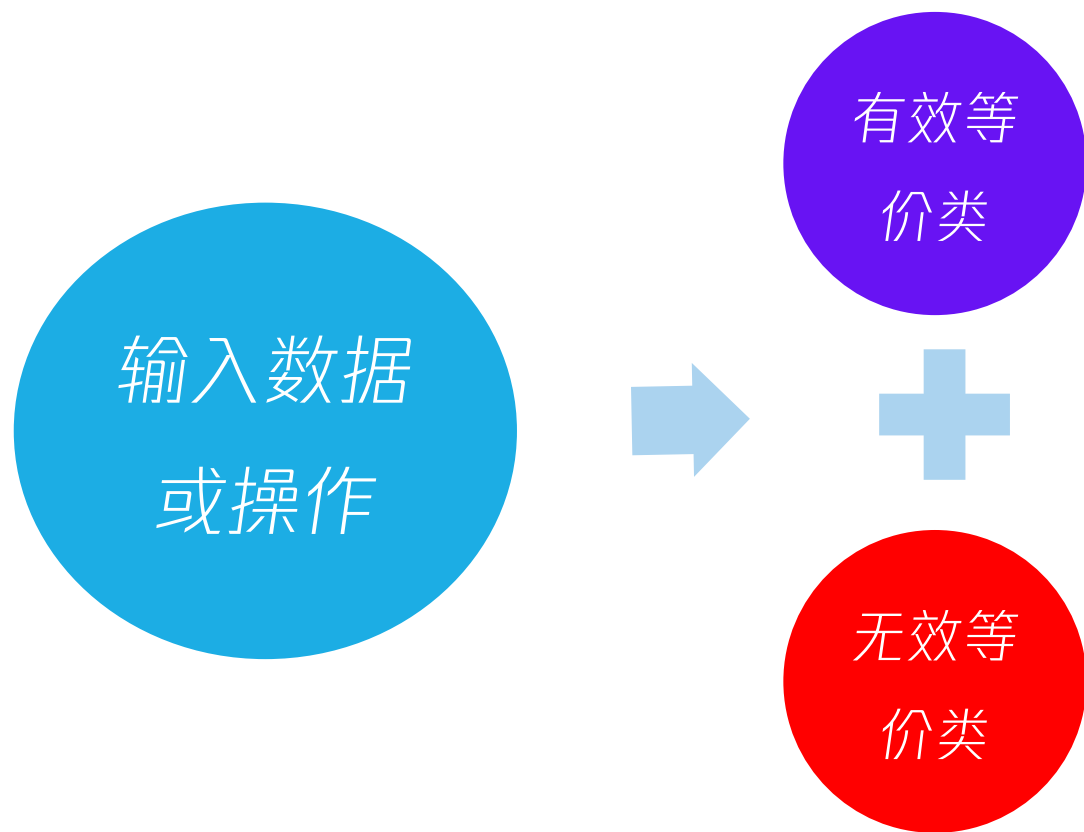
等价类划分法

根据软件需求规格说明中的相关内容将软件的输入域或操作分为多个子集合。

在划分子集合时，软件测试人员必须确保每个子集合中的各个数据输入或者操作系列对于揭露软件中的错误是等效的。

为了保证测试的完整性，软件测试人员在划分等价类时必须确保得到的等价类覆盖了软件的所有输入或者操作系列，并且各个等价类之间不相交。

7. 软件测试



设定的合理、有意义的输入数据或操作构成的集合。

不合理的、无意义的输入数据或操作系列构成的集合。

7. 软件测试

在进行等价类测试时，可以采用以下步骤：

- 等价类编号

为了便于管理和包含各个等价类，软件测试人员在开展测试之前必须为每个等价类规定一个唯一的编号；

- 为待测软件设计合理的测试用例，并开展测试

一个好的有效测试用例应当尽可能多地覆盖那些尚未被覆盖的有效等价类。软件测试人员实施了一个测试用例，就相当于验证了多个合理输入，减少软件测试的工作量。

- 为待测软件设计不合理的测试用例，并开展测试

与为有效等价类设计测试用例相反，为了便于发现错误，一个好的无效测试用例只能覆盖一个尚未被覆盖的无效等价类。

- 补充测试用例

软件测试人员可以结合上述原则为目标待测软件设计和补充测试用例，直到所有的等价类都被覆盖为止。

7. 软件测试

边界值分析法

将处理数据或者操作的边界情况纳入到测试用例中，将边界情况设计为测试用例参与测试。

如果在需求规格说明书中遇到第一个/最后一个、最小值/最大值、开始/完成，超过/在内、空/满、最短/最长、最慢/最快、最早/最迟、最大/最小、最高/最低、相邻/最远等关键字时，就需要结合边界值分析方法来设计测试用例。

7. 软件测试

因果图法

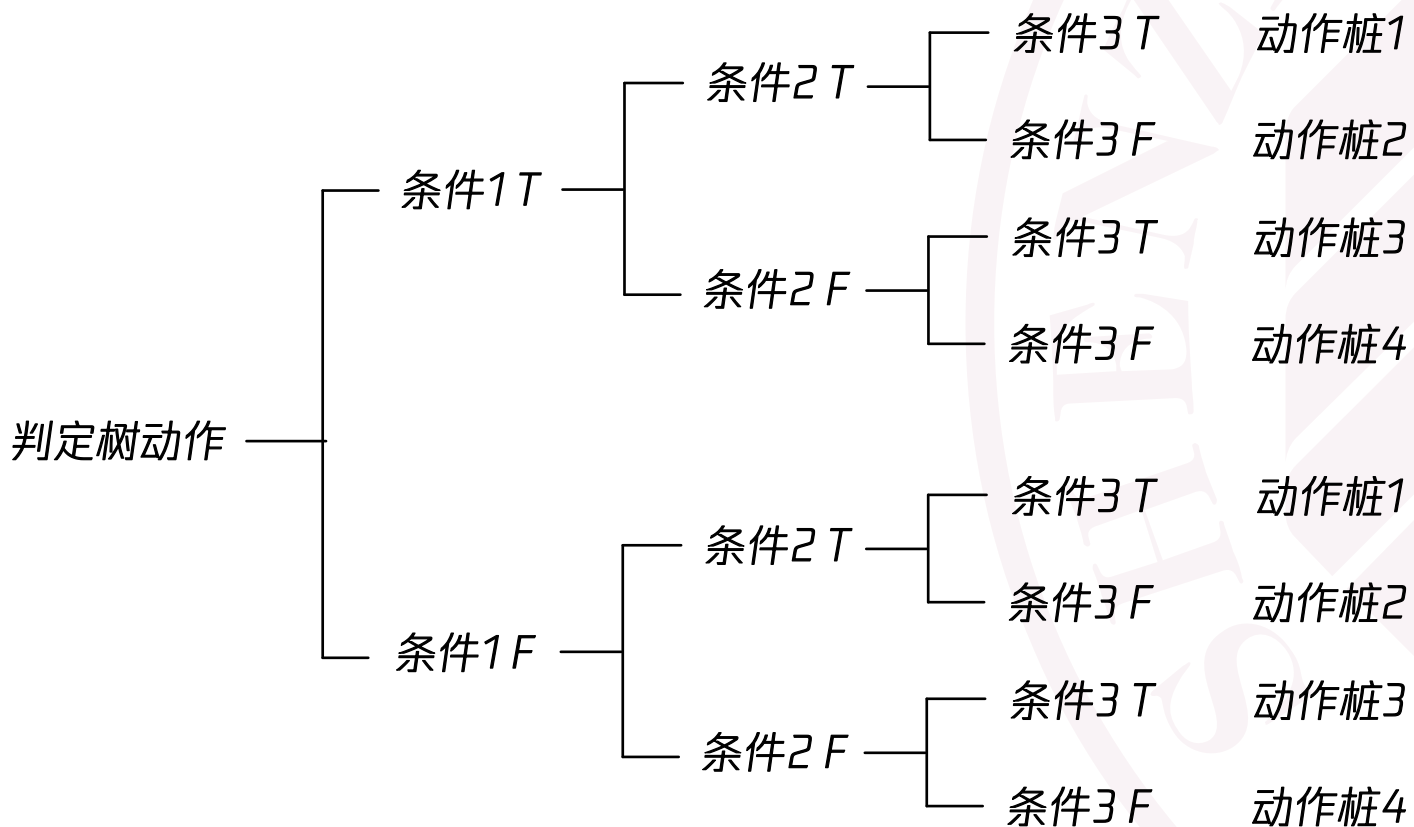
能够直观地表现输入和输出之间的对应关系。

	1	2	3	4	5	6
条件项1	T	T	T	F	F	F
条件项2	T	T	F	T	T	F
条件项3	T	F	T	T	F	F
动作桩1	√				√	
动作桩2				√		
动作桩3		√				
动作桩4						√
动作桩5			√			

7. 软件测试

判定表驱动法

工作原理与因果图表相类似，只是将因果图换为判定表。



7. 软件测试

错误推测法

软件测试人员也可以根据自己的直觉和测试经验列举出软件中可能出现的错误或者容易发生的错误。

错误推测法就是软件测试人员针对推测出的错误来设计测试用例的方法。

错误推测法中列举的错误必须来源于需求规格说明书，并对软件开发团队可能遗漏或忽略的需求内容，以及软件开发人员容易出现的错误来设计测试用例，确保软件对特定需求处理的正确性。

7. 软件测试

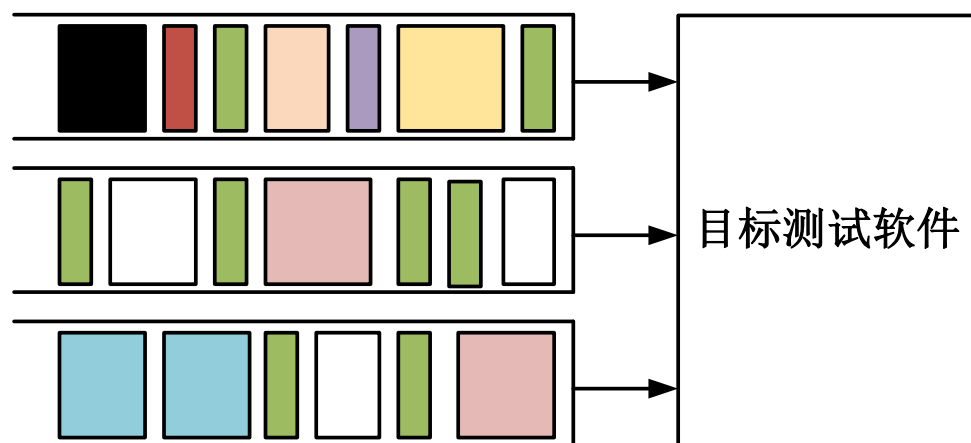
③ 性能测试

性能测试，也称为压力测试、强度测试和负载测试。

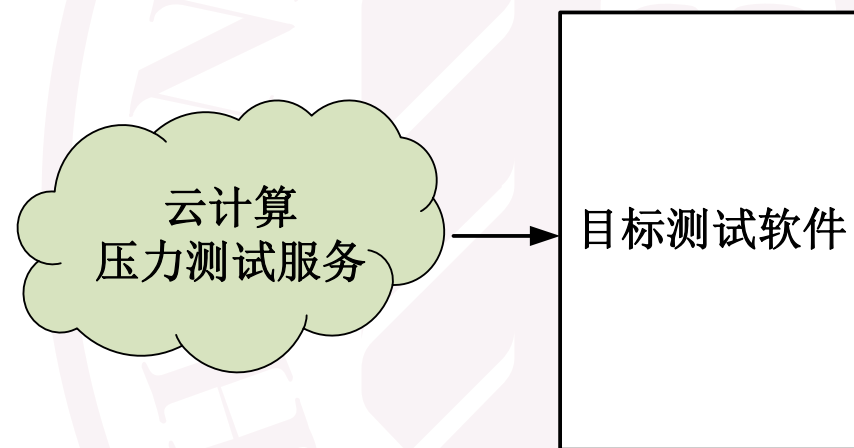
测试人员利用特定的测试工具或者开发的测试软件，依据软件需求规格说明书中的非功能性需求和约束条件，对目标软件不断施加压力的负荷测试过程。

7. 软件测试

测试人员通过模拟目标软件系统在实际应用或者用户使用过程中出现的真实负荷，检测目标软件系统的性能、可靠性、稳定性等指标是否满足要求。



通用压力测试



云压力测试

7. 软件测试

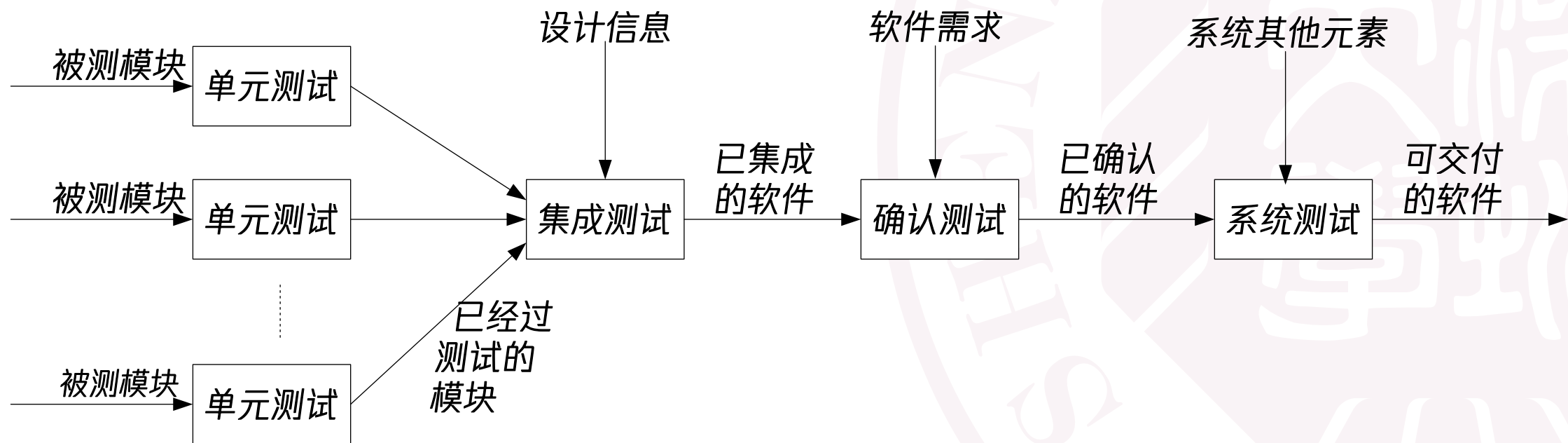
[2] 软件测试策略

通过多个不同的步骤，从各个层面对目标软件进行系统的、有计划的、有步骤的测试。



7. 软件测试

除了通用的软件测试步骤以外，也可以根据软件测试的不同阶段将软件测试过程顺序地分为单元测试、集成测试、确认测试和系统测试四个阶段。



7. 软件测试

① 单元测试

单元测试也称为模块测试，是针对目标软件的最小程序单元实施的测试工作。

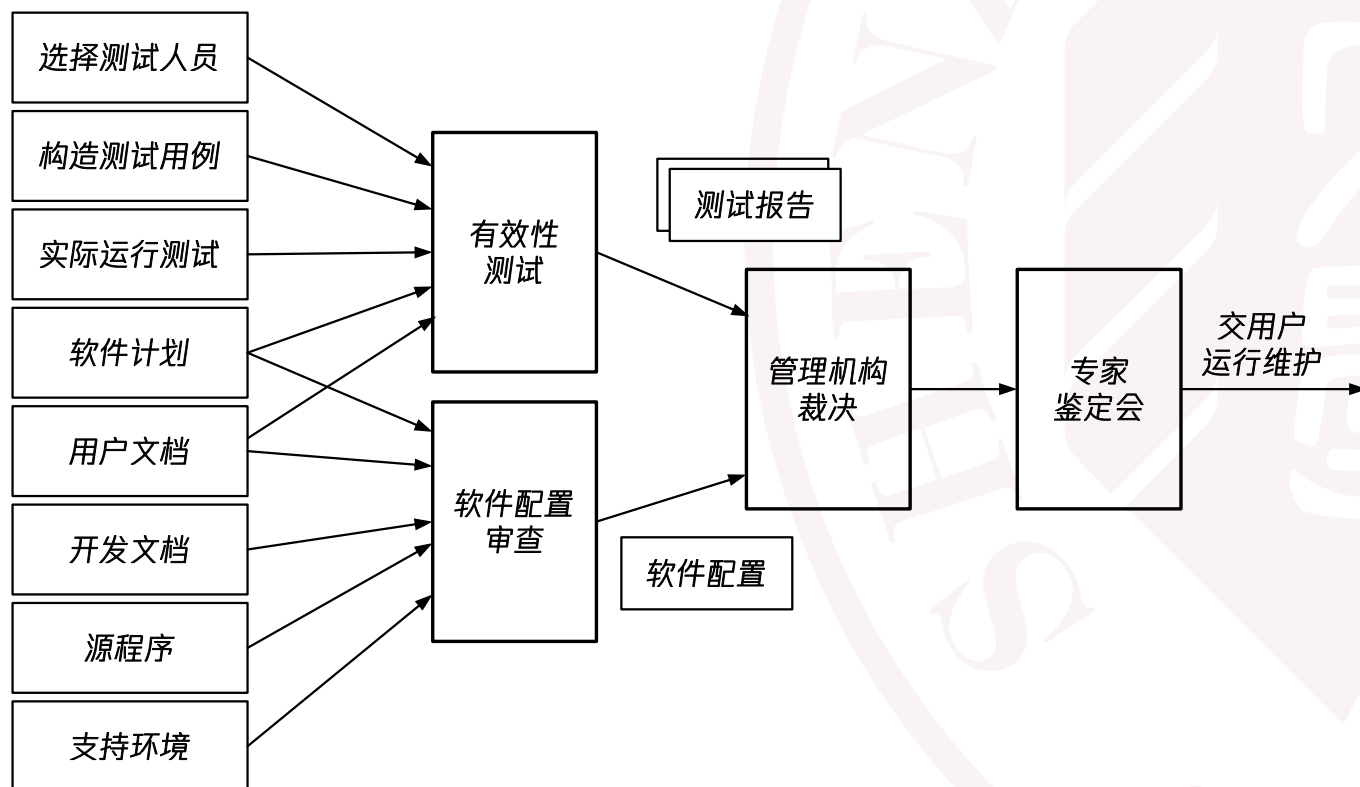
② 集成测试

集成测试也称为组装测试，是软件测试人员按照一定的集成策略，根据软件设计内容将各个经过测试的单元模块集成为目标软件的过程。

7. 软件测试

③ 确认测试

确认测试也称为验收测试，主要用来验证软件的有效性，即验证开发完成的软件系统在功能、性能，及其他特性方面是否与用户的要求一致。



7. 软件测试

完整的确认测试流程:



7. 软件测试

行业内广泛使用的确认测试方式：

✓ α 测试

软件测试人员根据用户设计的测试用例和操作习惯来检测软件系统中是否存在错误。 α 测试可以在开发环境中进行，也可以在模拟环境下进行。



7. 软件测试

✓ β 测试

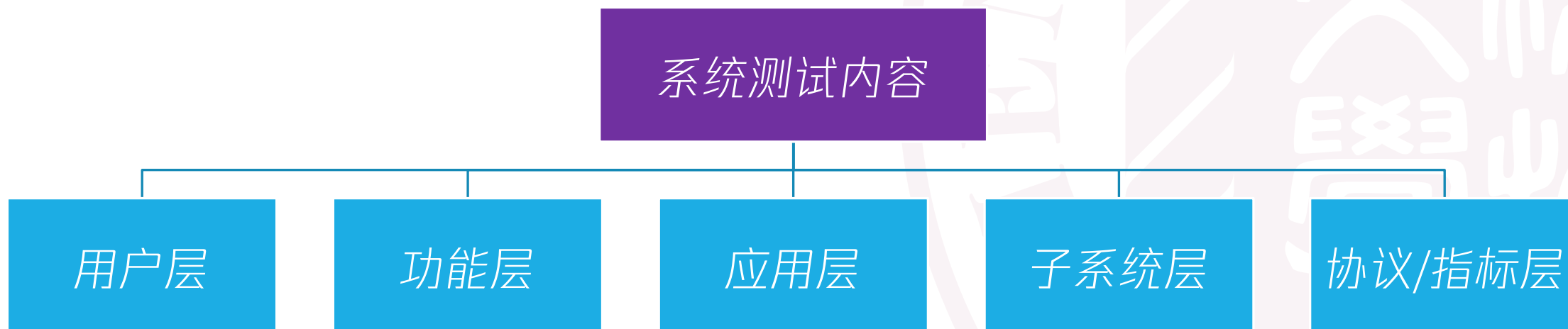
软件开发团队组织多个典型的用户，在实际工作环境〔非开发环境〕中对目标软件的 β 版本进行测试。

β 测试着重于产品的支持性评价，例如对软件文档、客户培训和支持产品生产能力等内容进行评价。只有当 α 测试达到一定的可靠程度时，软件开发团队才能申请开展 β 测试。

7. 软件测试

[3] 系统测试

将经过确认测试的软件系统与工作环境中的其他软、硬件环境集成起来，将完成的软件系统作为计算机系统的一部分，在实际的运行环境中开展严格、规范的测试。



7. 软件测试

具体测试内容:

易用性测试

功能测试

安全测试

压力测试

性能测试

容量测试

失效恢复测试

备份测试

兼容性测试

协议一致性测试

安装/卸载测试

7. 软件测试

[4] 软件测试步骤



7. 软件测试

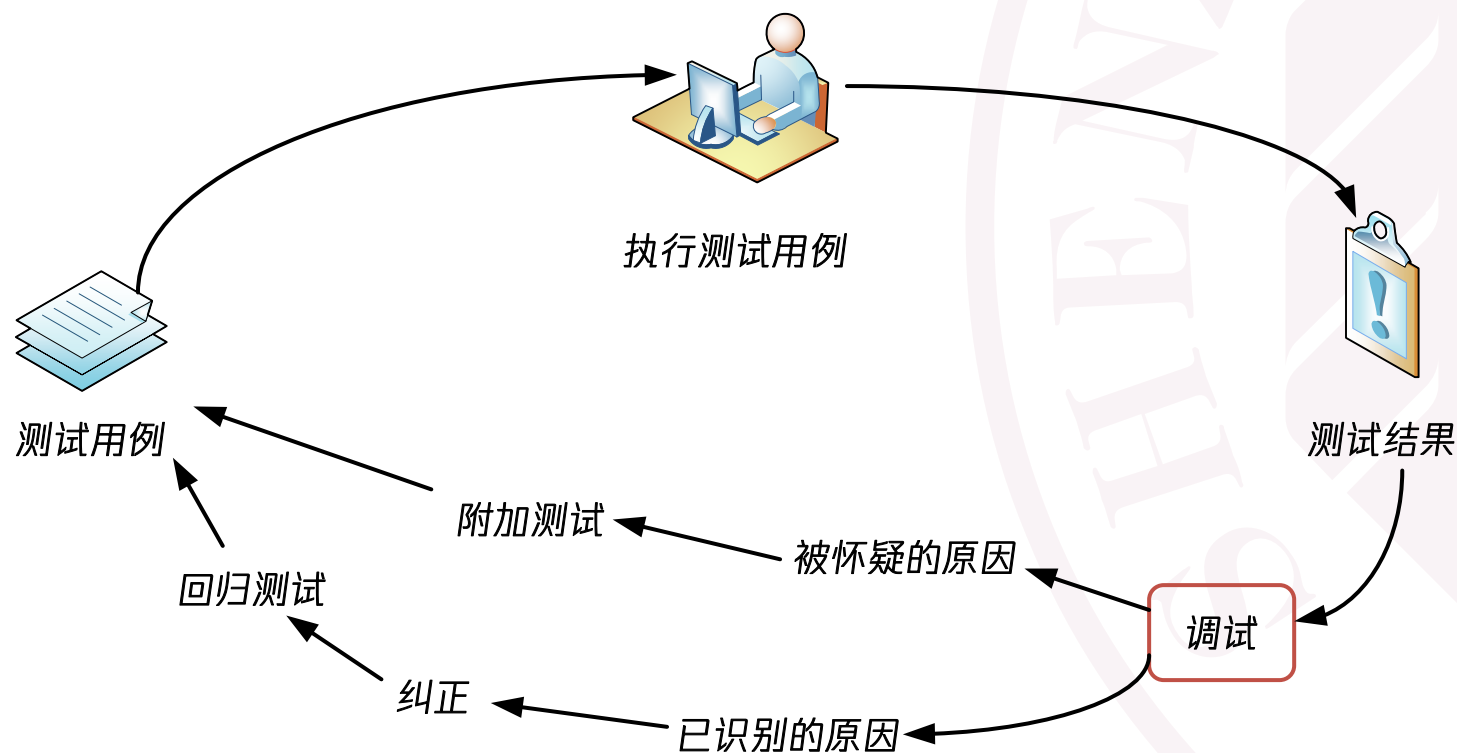
[5] 软件测试的原则

- ✓ 所有的测试内容都应该能够追溯到用户需求
- ✓ 测试计划必须先于测试
- ✓ 软件测试应从“小规模”开始，逐步转向“大规模”
- ✓ 对软件进行穷举测试是不可能的
- ✓ 测试只能查找出软件中的缺陷，不能证明软件中没有缺陷
- ✓ 在一个软件缺陷附近很可能还有更多的软件缺陷
- ✓ 避免使用相同的测试用例来测试软件
- ✓ 并非所有找到的软件缺陷都需要修复

7. 软件测试

[6] 软件调试Debug

软件开发人员通过分析软件的运行情况，确定导致软件出现错误或者缺陷的原因，并诊断和改正错误的过程。



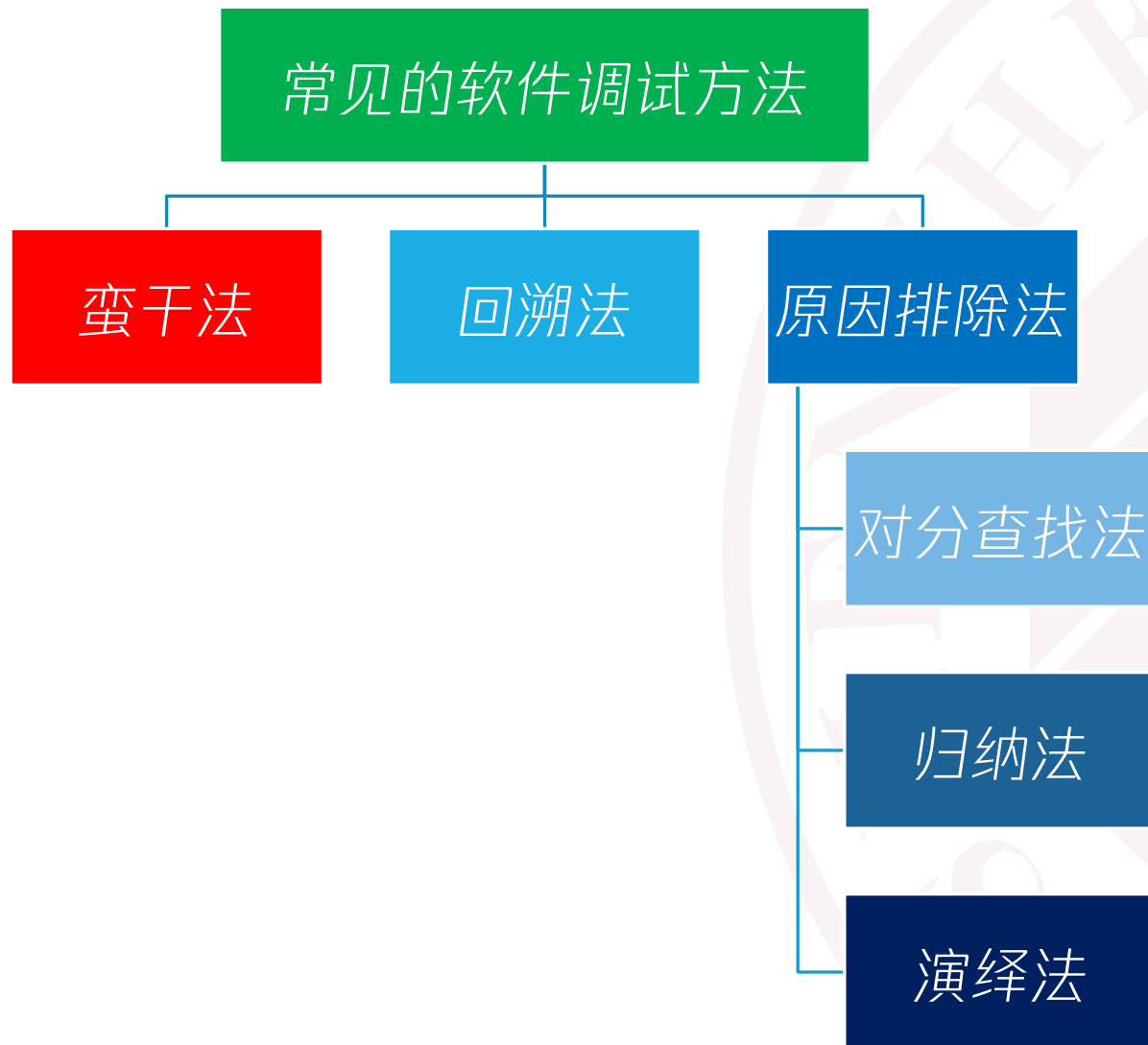
7. 软件测试

[6] 软件调试

软件调试困难的原因：

- 软件的错误表象与导致错误的原因所处的位置可能相距甚远；
- 当其他错误得到纠正时，当前的错误表象可能会暂时消失，但并未得到实际排除；
- 错误表象实际上是由一些非错误原因〔例如，舍入不精确〕引起的；
- 错误表象可能是由于一些不容易发现的人为错误引起的；
- 错误可能是由于时序问题引起的，与处理过程无关；
- 错误是由于难于精确再现的输入状态〔例如处理过程中的信号干扰〕引起的；
- 在软、硬件结合的嵌入式系统中，软件系统可能还会出现周期性错误。

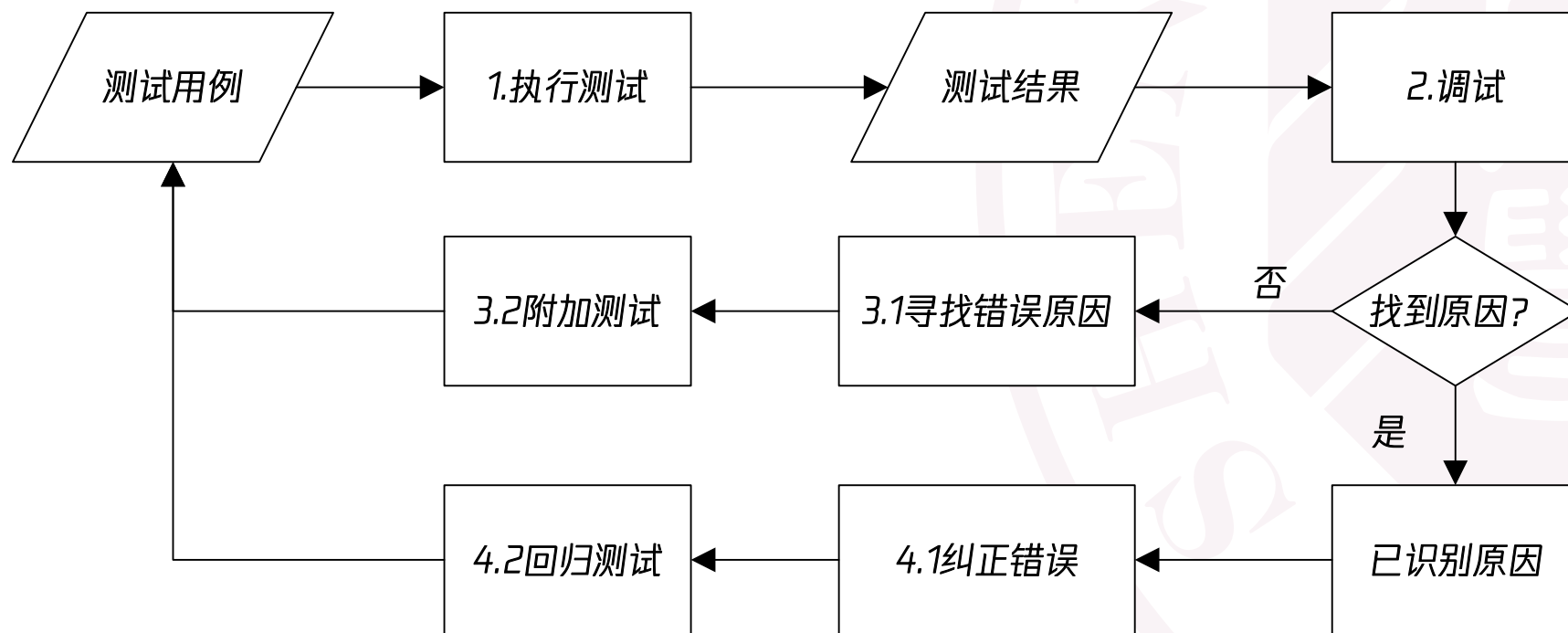
7. 软件测试



7. 软件测试

软件调试过程

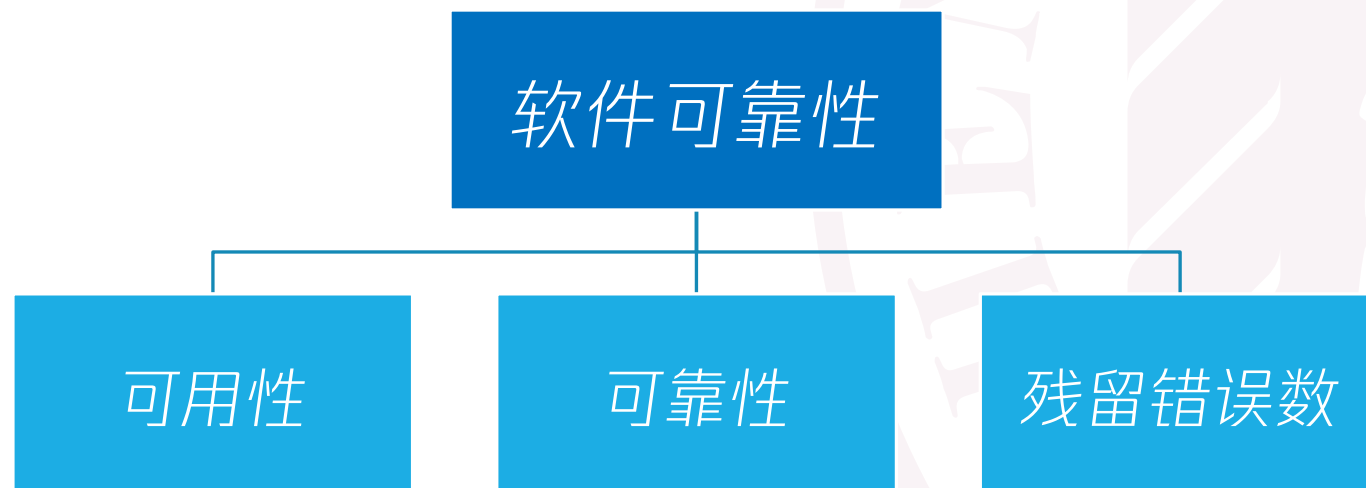
- 确定程序中可疑错误的确切性质和位置;
- 对程序〔设计, 编码〕进行修改、排除错误。



7. 软件测试

[7]软件可靠性

软件产品在规定的条件下和时间区间内完成规定功能的能力。



8. 软件维护

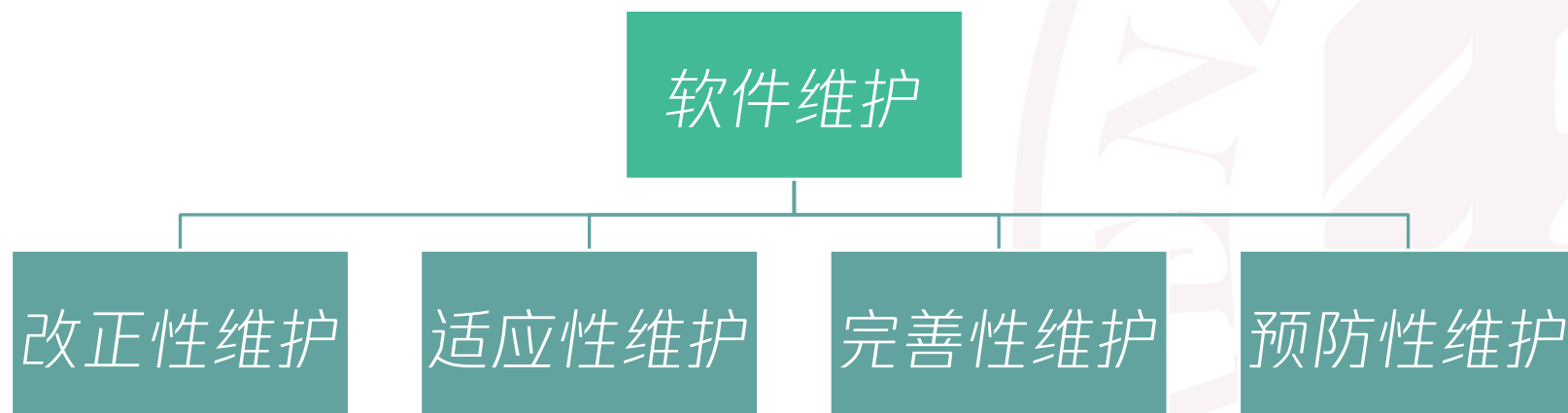
根据国标GB/T11457-95所述，
软件维护是指在软件产品交付使用
后，软件维护人员对软件进行修改，
纠正软件错误、改进软件性能和属
性，使产品能够适应环境的过程。

中华人民共和国国家标准	
软件工程技术语	
Software engineering terminology	
GB/T 11457—1995	
代替 GB 11457—89	
引言	
本标准结构如下：	
a. 词条按英文对应词字母顺序排列；	
b. 如果一个术语有一个以上的定义，则分别加以说明；	
c. 凡必要的地方用例子来说明定义；	
d. 为了说明本标准中一个术语与另一些术语的关系，使用了下述词语：	
——比较……：指补充性的术语；	
——与……相对照：指一个具有相反含义的或本质上不同意义的术语；	
——与……同义：指同义的术语；	
——参见……：指让读者参见推荐使用的或与之关系密切的术语。	
——还可参见……：指一有关术语。	
1 主题内容与适用范围	
本标准定义软件工程领域中通用的术语，适用于软件开发、使用维护、科研、教学和出版等方面。	
2 术语	
2.1 夭折，异常终止 abort	
在一过程完成之前被迫终止。	
2.2 绝对机器代码 absolute machine code	
每次使用时必须装入固定存储单元且不能再定位的机器语言代码。与 2.399 条相对照。	
2.3 抽象机 abstract machine	
a. 过程或机器的一种表示。	
b. 一个模块，它象一台机器那样处理输入。	
2.4 抽象 abstraction	
a. 对某一问题的概括。它抽取与某一特定目标相关的本质的内容而忽略非本质的内容。	
b. 形成上述抽象的过程。	
2.5 验收准则 acceptance criterion	
软件产品要符合某一测试阶段必须满足的准则，或软件产品满足交货要求的准则。	
2.6 验收测试 acceptance testing	
确定一系统是否符合其验收准则，使客户能确定是否接收此系统的正式测试。参见 2.381 条、2.497 条。	
2.7 可接近性 accessibility	
使组成软件的各部分便于选择使用或维护的程度。	
国家技术监督局 1995-05-04 批准	
1995-12-01 实施	
1	

国标GB/T11457-95

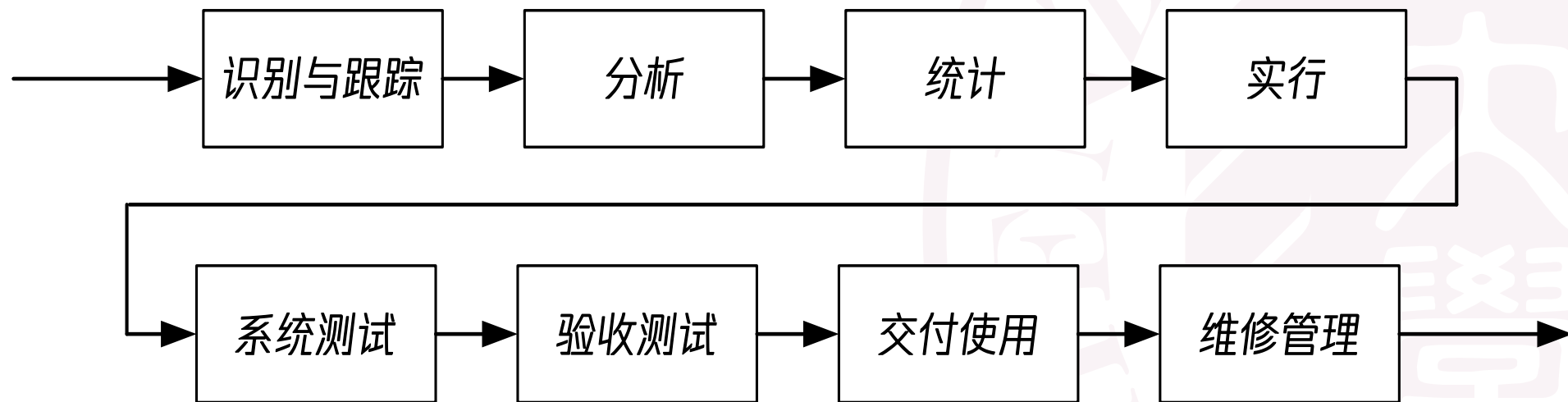
8. 软件维护

软件维护：在软件产品交付使用后，软件维护人员对软件进行修改，纠正软件错误、改进软件性能和属性，使产品能够适应环境的过程。



8. 软件维护

软件维护活动组成：

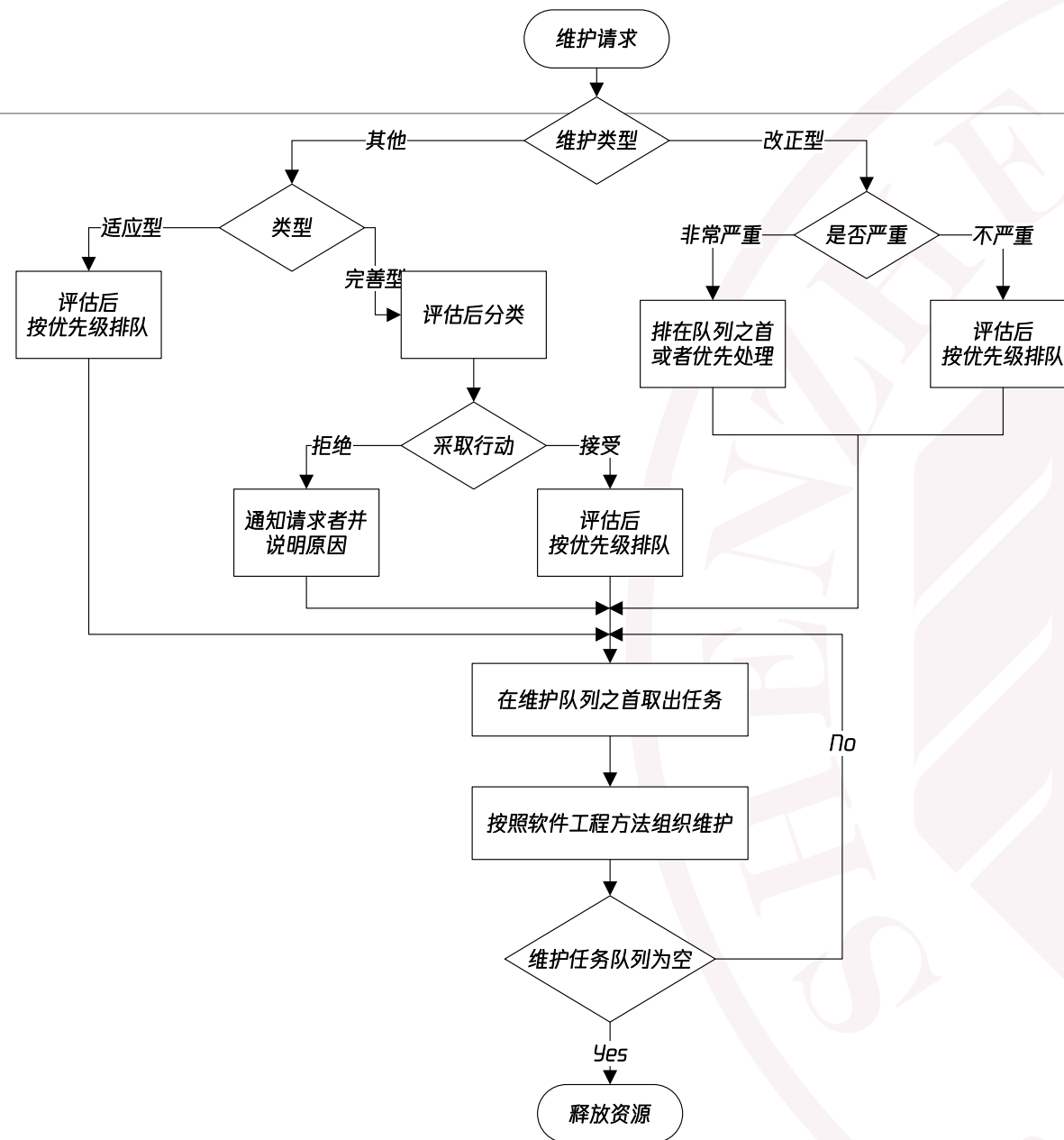


8. 软件维护

软件维护实施过程：



8. 软件维护



9. 小结



软件生命周期

9. 小结

软件生命周期作为软件开发和维护的一般过程，是所有软件开发方法的最基本原则。

在软件开发之前，软件的立项和可行性分析主要是对软件项目的意义和建设方案进行论证；

软件开发计划是结合软件项目的可用资源，对软件开发过程中的各个因素进行合理规划，统筹管理项目的实施过程。

9. 小结

随着软件开发技术的不断发展，各种软件开发技术层出不穷，并形成了结构化方法和面向对象方法。

各种软件开发方法均结合自身特点对软件生命周期中的需求分析、软件设计、软件实现和测试进行了补充和优化，使其可以满足相应的软件开发流程。

软件维护作为软件生命周期中的重要环节，可以持续地优化目标软件，充分发挥软件系统的价值。

谢谢大家!

课后思考:

1. 软件生命周期是什么? 你经历过软件生命周期的哪些阶段?
2. 所有软件开发方法在软件生命周期中完成的工作内容是相同的吗?
3. 软件生命周期里面涉及的内容是就是软件开发的全部吗?